

NOTE: For a full score of 100 points you should answer all 4 questions below.

Question 1. (Total 28 points)

In the following questions, you must write appropriate *pre*- and *post-conditions* for some informally defined mathematical operations using the JML specification language (i.e. write *requires-ensures* conditions).

You should try to avoid both over and under-specification, i.e. write all and only the constraints necessary for each pre- and post condition. Do not omit relevant constraints or add irrelevant ones. **Hint: if you write a text comment next to each formula explaining its intended meaning then your work will be clearer and you will be less likely to make mistakes.**

If you are unsure how any mathematical operation/method is defined you may look it up online or in a book. You do not have to provide Java code for any of the operations, only JML.

Detecting plagiarism is a special type of string-matching problem that can be defined in a top-down way using auxiliary methods. In the questions below you should try to re-use your earlier solutions in your later solutions to reflect this top-down design of a solution to the problem.

(a) (12 points) A string matching algorithm to find all copied text

```
int[] allCopiedSections(String target, source)
```

Input: A pair of strings `target` and `source`. Here `target` represents the work being analysed for plagiarism. The second parameter `source` represents another piece of text from which material might have been taken without authorization.

Output: An array `A` of positive integers of length $4n$ (for some $n \geq 0$). The array `A` contains consecutive 4-tuples of positive integers $A[4*i]$, $A[4*i + 1]$, $A[4*i + 2]$, $A[4*i+3]$, for each $(0 \leq i < n)$.

Each such 4-tuple in `A` represents two pairs of integer intervals $(A[4*i], A[4*i+1])$ which are locations in `target`, and $(A[4*i + 2], A[4*i + 3])$ which are locations in `source`.

The array `A` records these string interval endpoints so that:

$$A[4*i] < A[4*i + 1]$$

and

$$A[4*i + 2] < A[4*i+3].$$

The values of the `A[i]` are chosen so that substring of `target` starting at position `A[4*i]` and ending at position `A[4*i + 1]` is a case insensitive match onto the substring of `source` starting in position `A[4*i + 2]` and ending at position `A[4*i+3]`.

Furthermore, for the i -th 4-tuple, the `target` substring from `A[4*i]` to `A[4*i + 1]` is the longest possible string match we can find in `source`.

If the same `target` substring occurs twice or more in `source`, we only record one matching position, it doesn't matter which. To avoid trivial instances of matching, such as single letters, we assume that all the identified matching substrings have length at least 7.

(b) (5 points) A plagiarism estimator.

```
float percentagePlagiarism(String target, source)
```

Input: A pair of strings, where `target` represents the work being analysed for plagiarism and `source` represents another piece of text from which text material might have been borrowed without authorization.

Output: A `float` value representing the total percentage of all text in `target` for which we can find plagiarized text taken from `source`.

(c) (11 points) Assume that `percentagePlagiarism()` is implemented using `allCopiedSections()`. Explain why the requirements in part (a) are incomplete by designing a pair of test case values (`source =x`, `target =y`) that satisfy the preconditions of parts (a) and (b) for which `percentagePlagiarism()` returns the wrong percentage figure of plagiarized text. You should clearly describe what features of your test case break the postcondition by predicting from it what the correct percentage output should be from `percentagePlagiarism()`. What classical type of coding error is this, and why?

Question 2. (Total 10 points)

Compiler testing is an important subject in software testing, since if a programming language compiler has errors then these could potentially infect any software that it generates. One aspect of compiler functionality is *parsing* code according to some *grammar* rules which define the programming language syntax. Language syntax is often specified using one or more *regular expressions*.

Consider the following meta-compiler method

```
boolean parse(String s, RegularExpression A)
```

which takes as input a string s and a regular expression A (over the same alphabet) and returns `true` if the string s matches the regular expression A otherwise `false`.

Recall (or look up) that regular expressions over an alphabet Σ (such as the ASCII character set) are defined starting from

(i) string constants " $c_1, \dots, c_n$ " (where $c_i \in \Sigma$ is a character symbol) and n may be 0 (i.e. the empty string);

(ii) the empty set symbol $\emptyset$;

using the operators:

(iii) concatenation ($\text{Exp1} \cdot \text{Exp2}$);

(iv) alternation ($\text{Exp1} + \text{Exp2}$);

(v) Kleene star (Exp1^*).

(i) **(6 points)** Write down six different metamorphic equations that should be satisfied by the method `parse(.)` for any character set Σ . (By *different*, we mean here that no equation is just a special case of another equation.)

(ii) **(4 points)** Assume that Σ is the ASCII character set. Choose any four of your metamorphic equations derived for Part (i) and give a seed test case and the transformed test case for each equation.

Question 3. (Total 28 points)

Consider the UML state machine diagram for a *simple pocket calculator* given in Figure 1 below.

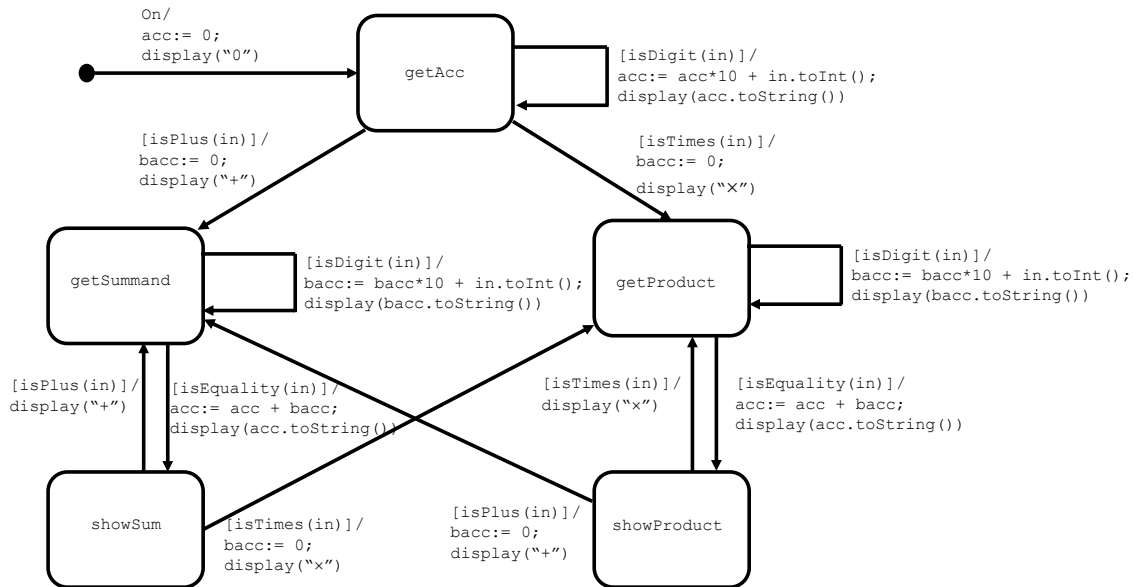


Figure 1. UML state machine for a simple pocket calculator

- (i) **(2 points)** Write a set of LTL trap properties which will give 100% node coverage.
- (ii) **(6 points)** Write a set of LTL trap properties which will give 100% edge coverage.
- (iii) **(5 points)** Identify two transitions in Figure 1, which clearly have errors, and explain what the errors are and what problems with calculations they would cause.
- (iv) **(10 points)** Discuss whether a test suite which gives 100% node coverage or a test suite giving 100% edge coverage (by implementing your trap requirements in parts (i) and (ii)) would discover the errors you identified in part (iii).
- (v) **(5 points)** Write LTL requirements which capture the correct behavior needed for calculation, and which the errors that you identified in part (iii) are violating.

Question 4. (Total 34 points)

Array access out-of-bounds errors (underflow and overflow) are a well-known and common type of programming error. Harry Hopeful wanted to rigorously unit test the following obscure piece of Java code using logic coverage:

```

1. myCode(int x, y): int
2. {
3.     int[] A = {2, 3, 4, 5, 6, 7, 8, 9, 10};
4.     int a;
  
```

```

5.      a = 0;
6.      if (x < 0) then return a else
7.      {
8.          if ((y>=0) | (x>9)) then
9.          {
10.             a = A[x+y];
11.             a = a + A[y];
12.             return a;
13.          } else
14.          {
15.             return A[0];
16.          } ;
17.      } ;
18. }

```

- (a) (2 points)** Draw a condensation graph for the routine `myCode`.
- (b) (6 points)** Discuss what kinds of execution errors can occur on line 10 of the code and, in each case, write a test case that would uncover the kind of error you identify.
- (c) (8 points)** Using the method of symbolic evaluation write a constraint table which shows the precise logical conditions on all variables sufficient and necessary to arrive at line 10. Hence derive sufficient and necessary logical conditions on the values of `x` and `y` which lead to: (i) underflow and (ii) overflow of the array access on line 10. Use Boolean and arithmetic logic to simplify your two formulas as much as possible.
- (d) (4 points)** Can you identify any other errors in the program? Would these errors be identified by your test cases for part (b)?
- (e) (6 points)** By finding necessary conditions on the program clauses that give rise to over- and underflow on line 10, show that there exists a set `S` of logic coverage test requirements which gives 100% predicate coverage (PC) of `myCode` but for which no test suite `T` satisfying `S` finds any errors on line 10.
- (f) (8 points)** In a similar way to Question 4.e, show that there exists a set `S` of logic coverage test requirements which gives 100% clause coverage (CC) of `myCode` but for which no test suite `T` satisfying `S` finds all the errors on line 10.